PORTING & DEPLOYMENT BLUEPRINT

# Caelestia on Parrot OS

A production-grade porting strategy and installer for the four-component Caelestia desktop ecosystem — C++ backing library, QML shell, TypeScript integrations and Python CLI — onto Parrot OS Home (Debian 13) with zero contamination of base system libraries and fully reproducible, rollback-safe deployment.

Linux Release Engineering

Parrot OS Home · Debian 13 “trixie” · /opt isolation architecture

NOVEMBER 2026 · PIN SET: SHELL V1.1.1 · QUICKSHELL V0.2.1

# Table of Contents


**1. Executive Summary** 2

**2. Ecosystem Anatomy and Runtime Topology** 3

2.1 How the components actually communicate . . . 3

**3. Cross-Distribution Conflict Analysis** 5

3.1 The Qt 6.9 wall (the decisive constraint) . . . 5

3.2 Quickshell: private build at a tested commit . . . 6

3.3 Python: PEP 668 makes the venv mandatory . . . 6

3.4 Node.js: a pinned, content-addressed layer . . . 6

3.5 Multiarch paths and the QML namespace . . . 7

3.6 The unportable remainder . . . 7

**4. Target Architecture under /opt/caelestia** 8

4.1 The five system touchpoints . . . 8

4.2 Environment mechanics . . . 9

**5. Toolchain: CMake, GCC and Multiarch** 9

5.1 Per-component build flags . . . 10

5.2 Multiarch resolution and version gates . . . 11

**6. Package Mapping: Arch to Debian 13** 12

**7. Version Pinning and Long-Term Stability** 13

7.1 Drift detection instead of holds . . . 14

**8. Orchestration Runtime and Failure Semantics** 14

**9. Security and Supply Chain** 16

**10. Verification and Diagnostics** 17

10.1 Static checks (no graphical session needed) . . . 17

10.2 Live smoke test (from inside the Hyprland session) . . . 17

**11. Operational Runbook** 18

11.1 Applying the dotfiles (deliberately manual) . . . 19

11.2 Upgrading the deployment . . . 19

11.3 Uninstall . . . 19

11.4 The private-Qt future option . . . 19

**Appendix A: install.sh Usage Reference** 20

**Appendix B: Lock File and File Inventory** 21

# 1. Executive Summary

This blueprint specifies how to move the Caelestia desktop ecosystem - a four-component stack built on Hyprland and Quickshell - from its native Arch Linux habitat onto Parrot OS Home, Debian's security-conscious, apt-managed distribution. The port must satisfy three hard constraints simultaneously: maximum long-term stability against future Parrot upgrades, a deployment that behaves identically on every machine it touches, and zero modification of the base system's libraries. Those constraints are in natural tension with Caelestia's rolling-release origins, and resolving that tension is the core engineering problem of this document.

The strategy that resolves it has two halves. First, a frozen, mutually-tested pin set: shell v1.1.1 (the last release whose CMake accepts Qt 6.8), quickshell v0.2.1 (exactly one commit after the commit pinned by shell v1.1.1's own flake.lock), the CLI commit 8bc7e49 that the same lock file records, a same-era dotfiles commit, and the current lib HEAD. Second, a strict isolation architecture: every artifact the ecosystem produces lives under /opt/caelestia, each language runtime gets its own sandbox (a Python venv, a pinned Node.js layer, a private CMake prefix), and the installer touches the base system in only five documented, reversible files.

Two facts discovered during the portability analysis drive most of the design. The first is that Caelestia's shell has required Qt 6.9 or newer since v1.2.0 (September 2025), while Parrot OS 7 is Debian 13 and ships Qt 6.8.2 - so current shell releases cannot compile against Parrot's system Qt at all, and a frozen era is the only honest option short of building a private Qt (rejected here as a maintenance liability). The second is that the ecosystem's cross-language wiring is looser than it looks: the CLI speaks to the shell through quickshell's own IPC command interface, not through linked code, which lets each component be built, upgraded and rolled back independently.

> **The zero-contamination contract.** Nothing is ever written into /usr, /usr/lib or /usr/local/lib; no system package is held, downgraded or patched; the externally-managed system Python is never modified. The complete system-level footprint is five files under /usr/local/bin, /etc/profile.d, /etc/environment.d and /etc/sudoers.d - all tracked, all removable with one command.

The deliverables that implement this blueprint are install.sh, a production-grade orchestrator with strict shell mode, signal-safe traps and automatic rollback; verify.sh, a post-install diagnostic that includes an apt-drift detector against a recorded manifest; and this document, which records the reasoning so future maintainers can change the design deliberately instead of discovering it.

# 2. Ecosystem Anatomy and Runtime Topology

Caelestia is not an application but a small society of cooperating processes. Understanding who talks to whom - and through which mechanism - is a precondition for isolating it correctly, because the isolation boundaries must never cut a live wire. This chapter maps the four components, their build systems and their runtime conversations.

| Component | Language | Build system | Installed as | Role |
|---|---|---|---|---|
| lib | C++20 / Qt6 | CMake + Ninja (min 3.25) | libcaelestia.so + Caelestia.Ipc QML module + theming helper | Backing library for the plugin ecosystem: config paths, IPC protocol, media analysis, scheme generation, Wayland capture |
| shell | QML + C++ plugin | CMake + Ninja (min 3.19) | Quickshell config tree + the 'Caelestia' QML plugin module | The visible desktop: bar, launcher, dashboard, notifications, lock screen |
| caelestia | TypeScript, Lua, CSS | npm / tsc / vsce (integrations) | Dotfiles tree + built .vsix extension artifacts | Session configuration for Hyprland and apps; VS Code and Firefox theme integrations |
| cli | Python 3.13 | hatchling + hatch-vcs | Console script 'caelestia' inside a virtualenv | User control surface: shell lifecycle, screenshots, wallpaper, schemes, recording |

*Table 1 - The four Caelestia components and their build products.*

## 2.1 How the components actually communicate

Field inspection of the pinned sources corrects a common misconception: the shell does not link against lib, and the CLI does not link against anything C++ at all. At the v1.1.1 era the CLI drives everything through quickshell's command interface: it launches the shell with qs -c caelestia -n and then issues requests such as qs -c caelestia ipc show or qs -c caelestia ipc call target message. The QML side registers the matching IPC handlers, so the transport is a local socket owned by quickshell itself. The CLI also shells out to session tools - grim, swappy, wl-copy, cliphist, fuzzel, sass, dconf - which is why those binaries appear in the dependency matrix of chapter 6.

The lib repository (created 2026-10-18) is the newest component: it extracts the shell's C++ utilities into a shared library so that third-party plugins can build against a stable API with find_package(caelestia). Its IPC module additionally defines a shell.sock endpoint under the XDG runtime directory for the upcoming plugin architecture, and its theme module ships a small root helper plus a sudoers rule that lets any local user apply Chromium policies and Papirus folder colors without a password prompt. The shell at our pinned era does not yet import these modules, so lib is deployed as platform infrastructure: present,

versioned, and ready for plugin development, without being load-bearing for the desktop itself.

The caelestia repository is the session layer: Hyprland configuration in Lua, fish/foot/fastfetch settings, and the two TypeScript integrations (VS Code, and later Firefox) that re-theme editors and browsers from the live Caelestia palette. The integrations are pure TypeScript builds with npm dependencies, which is precisely the kind of supply chain that must never be allowed to install into a global npm prefix on a shared system - chapter 4 shows how each build gets a hermetic Node layer instead.

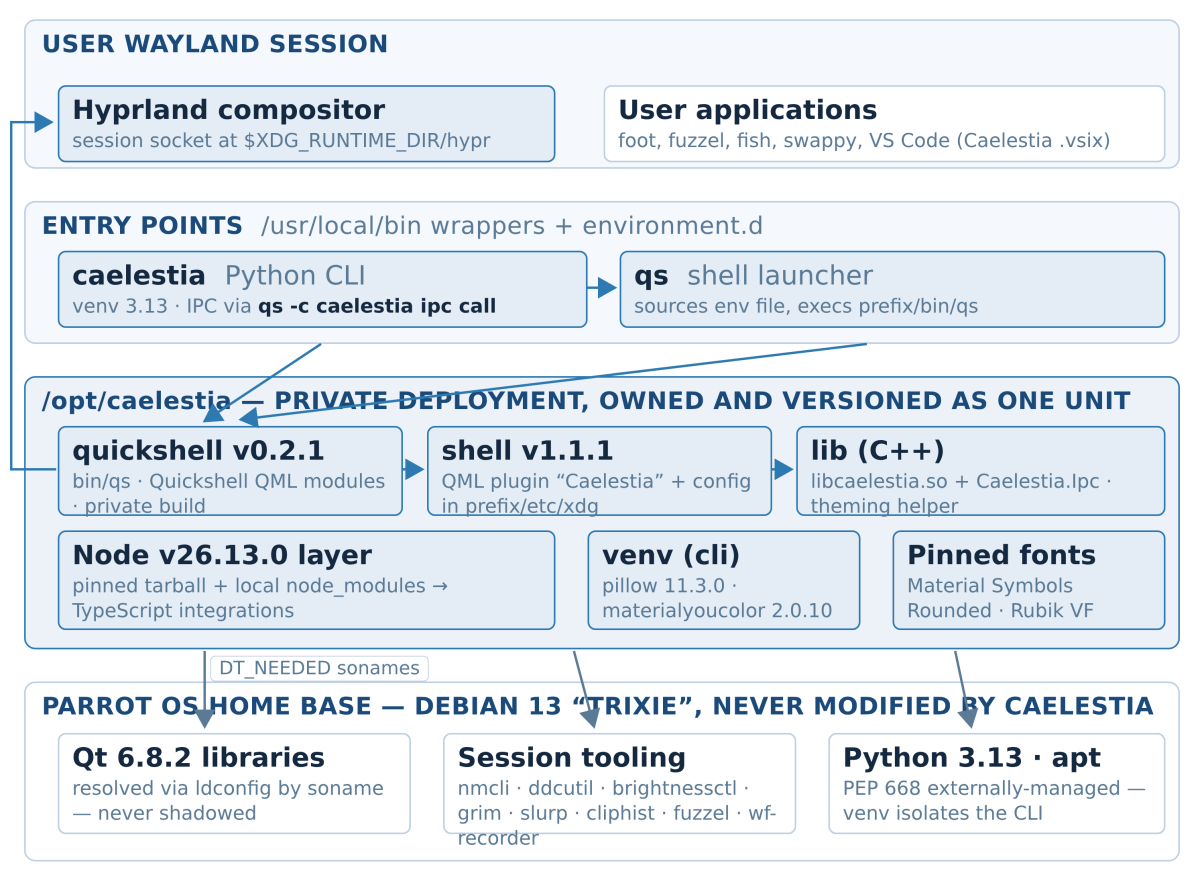



*Figure 1 - Runtime topology after installation. Blue arrows are Caelestia-owned control and data flows; gray arrows are dependency resolution against untouched system libraries.*

# 3. Cross-Distribution Conflict Analysis

Arch Linux and Debian optimize for opposite things. Arch rolls: every component is rebuilt against the newest library set within days of upstream releases. Debian freezes: trixie will ship Qt 6.8.x for its entire lifetime, receiving security backports but no ABI movement. A project developed on Arch therefore encodes assumptions - Qt versions, unified library paths, unsegmented Python and Node environments - that a Debian port must either satisfy or deliberately refuse. This chapter walks through each conflict class and the decision taken.

## 3.1 The Qt 6.9 wall (the decisive constraint)

Every Caelestia shell release since v1.2.0 executes qt_standard_project_setup(REQUIRES 6.9) in its plugin CMakeLists, which is a hard configure-time gate: the build refuses to start against Qt 6.8.2. Debian 13 ships Qt 6.8.2 in every qt6 package, verified against the trixie package index (qt6-base-dev 6.8.2+dfsg-9). The consequence is stark and worth stating plainly: current Caelestia cannot be compiled against Parrot's system Qt, and no flag, patch or environment variable changes that without forking upstream.

| Shell release | Date | Qt requirement | Builds on Parrot (Qt 6.8.2)? |
|---|---|---|---|
| v1.0.0 - v1.1.1 | 2025-08-29 to 09-03 | no version gate | Yes |
| v1.2.0 - v1.6.2 | 2025-09-14 to 2026-04-30 | REQUIRES 6.9 | No |
| v2.0.0 - v2.5.0 | 2026, current line | REQUIRES 6.9 + m3shapes + libqalculate/libpipewire/aubio/cava plugin deps | No |

*Table 2 - Qt requirement timeline of caelestia-dots/shell tags.*

Three escape routes exist and two are rejected deliberately. Pulling Qt 6.9 from Debian testing or a PPA would partially upgrade the most load-bearing GUI library on the system - the textbook recipe for dependency hell, and exactly what the zero-contamination contract forbids. Building a private Qt 6.9 under /opt is technically clean but converts a two-hour install into a multi-hour Qt compilation and adds a private 300-megabyte toolkit to maintain; it is documented as a future option (a one-paragraph sketch in chapter 11) rather than shipped. The route this blueprint takes is the third: pin the last Qt-6.8-buildable generation and freeze it as the compatibility contract, exactly as a Linux distribution would.

## 3.2 Quickshell: private build at a tested commit

Quickshell, the widget framework the shell renders through, is packaged in Debian only from v0.3.0 (trixie-backports) - eight months newer than the quickshell generation the pinned shell was written against, and carrying a major-minor QML API boundary. The shell's own flake.lock records the exact quickshell commit it was tested with: f7597cda, 2025-08-27. The release tag v0.2.1 (2025-10-11) sits exactly one commit after that pin, making it the closest real release to the tested state. The installer therefore builds quickshell v0.2.1 from source into the private prefix - which also guarantees that the shell's QML plugin compiles against matching headers, since Debian's binary package ships none.

One caveat is handled explicitly: quickshell v0.2.1's crash reporter links Google Breakpad, which Debian does not package. The installer configures -DCRASH_REPORTER=OFF, trading upstream crash telemetry for a dependency that actually exists on the target. Upstream's build documentation of that era claims Wayland support only for Qt 6.6/6.7; in practice Debian itself built quickshell 0.3.0 against Qt 6.8.2 for trixie-backports, and the v0.2.1 line was developed while Qt 6.8 was current on Arch. The post-install QML smoke test in chapter 10 is the final arbiter: if the private quickshell misbehaves on 6.8.2, verification fails loudly instead of the desktop failing silently.

## 3.3 Python: PEP 668 makes the venv mandatory

Debian 13 marks the system Python as externally managed: pip install into it refuses unless overridden with --break-system-packages, and Parrot's own tooling depends on that interpreter staying pristine. The CLI (hatchling-built, requiring Python 3.13, depending on pillow and materialyoucolor) is therefore installed into a dedicated virtualenv at /opt/caelestia/venv. The console script is exposed through a wrapper that sources the environment file, so users type caelestia and get the isolated interpreter. The two PyPI dependencies are pinned to the versions current when the CLI pin was made (pillow 11.3.0, materialyoucolor 2.0.10) so that upstream library drift cannot change color-scheme output on a system that has not otherwise changed.

## 3.4 Node.js: a pinned, content-addressed layer

Debian's nodejs 20.19 is serviceable, but the TypeScript integrations declare modern toolchains and the system package would put npm's global state under /usr/lib - precisely the contamination this port exists to avoid. The installer downloads the official Node v26.13.0 tarball, verifies it against a pinned SHA-256, and unpacks it under /opt/caelestia/node/v26.13.0 with a 'current' symlink. All npm activity runs with a cache, fund and audit configuration scoped to /opt, and node_modules live inside each integration's source tree. Nothing is ever installed with npm -g.

## 3.5 Multiarch paths and the QML namespace

Arch merged everything into /usr/lib; Debian splits native libraries into /usr/lib/x86_64-linux-gnu (or aarch64-linux-gnu, and so on). Caelestia's install destinations default to Arch-style paths, so the installer overrides every CMake install variable with prefix-relative Debian-style paths. The triplet is never hardcoded: it is resolved at runtime from dpkg-architecture, falling back to gcc -dumpmachine, falling back to uname -m mapping. Debian also splits Qt's QML modules into dozens of qml6-module-* runtime packages; the shell's import list at the pinned era (QtQuick, Controls, Layouts, Shapes, Effects, Window, Workerscript) was mapped one-to-one onto verified trixie packages so the QML engine finds every module without a single file outside /opt.

## 3.6 The unportable remainder

A final class of dependencies simply does not exist in Debian 13 and must be handled by policy rather than packaging. They are small, enumerable, and none of them blocks the core desktop experience.

| Arch dependency | Status in Debian 13 | Handling in this port |
|---|---|---|
| ttf-material-symbols-variable | not packaged | Pinned SHA-256 download from the upstream Google repository into the private prefix; fontconfig picks it up via XDG_DATA_DIRS |
| ttf-rubik-vf | not packaged | Same pinned-download treatment (icon and text fonts the shell hard-references by family name) |
| dart-sass | not packaged (sassc is) | Install sassc plus a three-line 'sass' shim in the prefix; the CLI's Discord theming call compiles unchanged |
| gpu-screen-recorder / wl-screenrec | not packaged | The era CLI already falls back to wf-recorder, which trixie ships; recording works, just via a different encoder |
| qt6-imageformats | only -doc packages exist | Documented limitation: wallpapers should be PNG/JPEG (no WebP or TIFF QML image plugins on trixie) |
| hyprland | trixie-backports only | Optional --with-hyprland stage installs from apt when available, otherwise prints exact backports instructions |
| google-breakpad | not packaged | quickshell built with -DCRASH_REPORTER=OFF (see 3.2) |

*Table 3 - Dependencies absent from Debian 13 and their explicit policies.*

# 4. Target Architecture under /opt/caelestia

The Filesystem Hierarchy Standard reserves /opt for add-on software packages - self-contained trees that a vendor or administrator owns outright. That is exactly the ownership model this port needs: one directory whose contents can be versioned, replaced atomically, snapshotted for rollback, and deleted without leaving residue. Debian's own packaging never touches /opt, and Parrot inherits that guarantee.

**Directory layout (the is resolved at install time)**

```
/opt/caelestia/
|-- env # environment contract (sourced by wrappers)
|-- lock.env # component pins + resolved SHAs (the contract)
|-- manifest.env # system snapshot at install time (drift ref)
|-- verify.sh # diagnostic suite (install.sh --doctor)
|-- prefix/ # CMake install root for ALL native parts
| |-- bin/qs, bin/sass-shim, bin/caelestia-theming-helper
| `-- lib// # multiarch: libcaelestia.so, qt6/qml/...
|-- venv/ # Python 3.13 virtualenv for the cli
|-- node/
| |-- v26.13.0/ # pinned, sha256-verified Node runtime
| `-- current -> v26.13.0
|-- src/ # pinned checkouts (lib, shell, cli, ...)
|-- build/ # ninja build dirs (discarded by rollback)
|-- backups/ # prefix-/, venv-/ rollback points
|-- logs/ # full install logs, one per run
`-- state/ # touchpoint registry, pip constraints
```

## 4.1 The five system touchpoints

A user-space desktop still needs to be reachable: binaries must be on PATH, login shells and systemd user sessions must inherit the environment, and the theming helper needs its sudo rule. Each of these is an additive file in a directory explicitly designed for local administrator additions; every one is tracked in a registry and removed by install.sh --uninstall. Nothing here can affect another package's files, because none of these paths are owned by dpkg.

| Touchpoint | Purpose | Removal |
|---|---|---|
| /usr/local/bin/caelestia | wrapper: sources env, execs the venv CLI | tracked, auto-removed |
| /usr/local/bin/qs | wrapper: sources env, execs prefix/bin/qs | tracked, auto-removed |
| /etc/profile.d/caelestia.sh | login shells inherit the environment | tracked, auto-removed |
| /etc/environment.d/50-caelestia.conf | systemd user sessions (Hyprland via uwsm or a display manager) inherit it | tracked, auto-removed |
| /etc/sudoers.d/caelestia-theming | passwordless run of the lib theming helper and papirus-folders only | tracked, auto-removed |

*Table 4 - Complete system-level footprint outside /opt (5 unique files; wrappers plus 3 integration files and the optional sudoers rule; the registry also records per-run copies).*

## 4.2 Environment mechanics

The env file is the single source of runtime truth. It prepends the private bin directories to PATH, points LD_LIBRARY_PATH only at the private multiarch directory (system libraries keep resolving through the standard ldconfig cache, so no global library path is polluted), sets QML2_IMPORT_PATH to the one private QML root shared by quickshell, the shell plugin and lib, and extends XDG_CONFIG_DIRS and XDG_DATA_DIRS so quickshell discovers the shell config and fontconfig discovers the pinned fonts. Wrappers source it and exec the real binary, which means the environment also applies to every child process the shell spawns.

Rollback points are the second half of the architecture. Before the first build stage mutates the prefix, the installer moves any previous tree to backups/prefix- and builds a fresh one in place - in place, rather than into a staging directory renamed later, because CMake bakes absolute RPATH and QML paths into its output and a rename would corrupt them. If any later stage fails, the EXIT trap restores the backup; on success, older backups are pruned to a bounded history of two, giving the administrator a one-command manual rollback window after every upgrade.

# 5. Toolchain: CMake, GCC and Multiarch

Every native component builds with the same CMake contract, so the flag rationale can be argued once and applied everywhere. The goals: deterministic release builds, hardening equal to Debian's own package defaults, and linkage that records the minimum set of dependencies so our objects can never accidentally couple to a system symbol they do not truly use.

**Shared CMake contract (identical for quickshell, lib, shell)**

```
cmake -S -B -G Ninja \
-DCMAKE_BUILD_TYPE=Release \
```

```
-DCMAKE_INSTALL_PREFIX=/opt/caelestia/prefix \
-DCMAKE_INSTALL_LIBDIR=lib/$DEB_HOST_MULTIARCH \
-DCMAKE_INSTALL_RPATH=/opt/caelestia/prefix/lib/$DEB_HOST_MULTIARCH \
-DCMAKE_BUILD_WITH_INSTALL_RPATH=ON \
-DCMAKE_CXX_FLAGS_RELEASE="-O2 -g -DNDEBUG \
-Wp,-D_FORTIFY_SOURCE=2 -fstack-protector-strong" \
-DCMAKE_EXE_LINKER_FLAGS="-Wl,--as-needed -Wl,-z,relro -Wl,-z,now" \
-DCMAKE_SHARED_LINKER_FLAGS="-Wl,--as-needed -Wl,-z,relro" \
-DDISTRIBUTOR="Parrot-Caelestia-Port"
```

- **-O2 instead of upstream -O3:** -O2 is what Debian itself builds Qt and KDE with; it is the best-tested optimization level on this exact toolchain, and Qt-heavy QML plugin code has a history of exposing compiler edge cases at -O3. Predictability beats the last two percent.
- **Hardening:** _FORTIFY_SOURCE=2 and -fstack-protector-strong mirror Debian's dpkg-buildflags defaults and are safe in shared objects; relro/now at link time give full RELRO on executables.
- **-Wl,--as-needed:** the recorded DT_NEEDED entries shrink to what the code actually references. This is the technical core of "not breaking system symbols": our libraries declare only real dependencies, so a future system library change can only affect us through a symbol we genuinely use.
- **RPATH policy:** one absolute RPATH into our own prefix and nothing else. We never rpath into /usr (which could shadow system resolution), never set LD_LIBRARY_PATH globally, and never install a file whose soname collides with a system library.
- **Multiarch:** CMAKE_INSTALL_LIBDIR is the dynamically resolved triplet, so libcaelestia.so lands in prefix/lib/x86_64-linux-gnu exactly where Debian convention puts native libraries.

## 5.1 Per-component build flags

On top of the shared contract, each component needs a small set of specific decisions. They are all pinned in install.sh; the table records why each exists so that a future maintainer can distinguish a load-bearing flag from cargo cult.

| Component | Flag | Reason |
|---|---|---|
| lib | -DVERSION=0.0.0<br>-DGIT_REVISION= | lib is a rolling repo with no git tags; its CMake derives the version from git describe and would hard-fail on the pinned shallow checkout. The overrides keep the build deterministic and land in the component manifest lib installs. |
| lib | -DINSTALL_SUDOERS=ON<br>-DSUDOERS_DIR=/etc/sudoers.d | The theme helper needs a real sudoers drop-in to function; the installer renames the produced file (sudo ignores names containing a dot), sets 0440/root and validates with visudo. |
| quickshell | -DINSTALL_QMLDIR=/lib//qt6/qml | Routes quickshell's own QML modules into the same single import root as the Caelestia modules, so one env var covers everything. |
| quickshell | -DCRASH_REPORTER=OFF | v0.2.1 crash handling links Google Breakpad, not packaged in trixie (see 3.2). |
| shell | -DINSTALL_QMLDIR=lib//qt6/qml<br>-DINSTALL_LIBDIR=lib//caelestia<br>-DINSTALL_QSCONFDIR=etc/xdg/quickshell/caelestia | v1.1.1's defaults are Arch-style paths (usr/lib/qt6/qml, relative to an install prefix of /). All three are redirected to prefix-relative Debian-style destinations. |
| cli | SETUPTOOLS_SCM_PRETEND_VERSION=1.0.0 | hatch-vcs derives the version from git tags; the pinned commit-only checkout has none, so the version is fixed explicitly. |

*Table 5 - Component-specific build decisions.*

## 5.2 Multiarch resolution and version gates

The installer resolves the triplet once, early, and fails loudly if the platform is unsupported; the same value then flows into every CMake invocation, the env file and the verification script.

**Triplet resolution (most authoritative source first)**

```
resolve_arch() {
dpkg-architecture -qDEB_HOST_MULTIARCH \
|| gcc -dumpmachine \
|| case "$(uname -m)" in
x86_64) echo x86_64-linux-gnu ;;
aarch64) echo aarch64-linux-gnu ;;
esac
}
```

Version gates run in preflight and again after the apt stage: Qt 6.7 minimum for lib's qt_standard_project_setup gate (Parrot's 6.8.2 passes), wayland-protocols 1.37 minimum for lib's screenshot module (trixie ships 1.44), Python 3.13 for the CLI (trixie ships 3.13.5), and CMake 3.25 for lib (trixie ships 3.31). If Parrot's next major release bumps Qt past 6.9, the gate notices and prints that newer Caelestia pins have become buildable - the pin set stays frozen until an administrator deliberately updates it.

# 6. Package Mapping: Arch to Debian 13

Every dependency claimed by the pinned Caelestia sources was checked against the authoritative Debian 13 package index rather than against memory or third-party trackers. The result splits into three groups: packages that exist and are installed, packages that exist but need a policy decision, and packages that do not exist (covered in table 3). The installer installs build dependencies with --no-install-recommends so the resulting machine state is exactly the enumerated set.

| Role | Arch name | Debian 13 name(s) | Verified |
|---|---|---|---|
| Qt build | qt6-base, qt6-declarative | qt6-base-dev, qt6-declarative-dev (+ -private-dev for quickshell) | 6.8.2+dfsg |
| Qt extras | qt6-shadertools, qt6-multimedia | qt6-shadertools-dev, qt6-multimedia-dev | 6.8.2 |
| Wayland | wayland, wayland-protocols | libwayland-dev, wayland-protocols | 1.23.1 / 1.44 |
| quickshell deps | cli11, spirv-tools, jemalloc, ... | libcli11-dev, spirv-tools, libjemalloc-dev, libdrm-dev, libgbm-dev, libegl/gles/vulkan-dev, libxcb1-dev, libpipewire-0.3-dev, libpam0g-dev, libpolkit-*-dev, libglib2.0-dev | present |
| lib media | ffmpeg | libavcodec-dev, libavformat-dev, libavutil-dev, libswscale-dev | 7.1.5 |
| QML runtime | (bundled in qt6) | qml6-module-qtquick{,-controls,-templates,-layouts,-shapes,-effects,-window}, qml6-module-qtqml-workerscript, qt6-svg-plugins, libqt6svg6 | 6.8.2+dfsg |
| Session tools | networkmanager, ddcutil, brightnessctl, ... | network-manager, ddcutil, brightnessctl, lm-sensors, libnotify-bin, libglib2.0-bin, fish, dconf-cli | present |
| Screenshot etc. | grim, slurp, swappy, wl-clipboard, cliphist, fuzzel | same names | present |
| Recording | gpu-screen-recorder / wl-screenrec | wf-recorder (CLI fallback), see table 3 | present |
| Python | python 3.13 | python3.13, python3-venv, python3-pip | 3.13.5 |
| Icons / fonts | papirus-icon-theme, ttf-jetbrains-mono-nerd | papirus-icon-theme, fonts-jetbrains-mono | present |

*Table 6 - Dependency mapping verified against the trixie Packages index (November 2026).*

Two apt policy decisions are deliberate. First, no package is held with apt-mark hold: security updates to Qt or glibc must keep flowing to a security-focused distribution, and the deployment is designed so that it absorbs them (our binaries bind system libraries by soname, and Qt guarantees binary compatibility across patch releases). Second, the Hyprland compositor is not silently installed: it lives in trixie-backports on plain Debian, and enabling an extra repository is an administrative decision the installer only assists with (--with-hyprland attempts apt and otherwise prints the exact backports commands).

# 7. Version Pinning and Long-Term Stability

The pin set is the compatibility contract of the whole deployment. It was not assembled by taking newest-of-everything; it was assembled by asking which exact revisions upstream itself tested together, and then freezing that answer into lock.env, a plain shell-sourceable file an administrator can read, diff and edit.

| Component | Pin | Date | Why this revision |
|---|---|---|---|
| shell | v1.1.1 (tag) | 2025-09-03 | Last release with no Qt version gate; builds on Parrot's Qt 6.8.2 |
| cli | 8bc7e49 (commit) | 2025-08-30 | The exact CLI commit recorded in shell v1.1.1's flake.lock |
| quickshell | v0.2.1 (tag) | 2025-10-11 | One commit after f7597cda, the flake.lock-tested quickshell pin; same 0.2.x QML API generation |
| caelestia (dots) | 1a69a48 (commit) | 2025-09-01 | Same-era dotfiles; contains the VS Code TS integration, predates the Firefox one |
| lib | d8f3b72 (commit) | 2026-11-05 | Current HEAD of the (tagless, 3-week-old) backing library; only Qt 6.7 gate, independent of shell era |
| Node.js | v26.13.0 tarball | current LTS | SHA-256 pinned from nodejs.org; runs tsc/vsce for the TS builds |
| PyPI deps | pillow 11.3.0, materialyoucolor 2.0.10 | era | Versions current when the CLI pin was made; prevents scheme-generation drift |

*Table 7 - The pin set and the lineage of each choice.*

### 7.1 Drift detection instead of holds

Long-term stability on a rolling base system is achieved by observation, not by freezing apt. At install time the installer writes manifest.env recording the exact versions of the system packages the deployment was validated against (libqt6core6t64, wayland-protocols, pipewire, network-manager and friends). verify.sh compares the live system against that manifest on every run: a changed version is reported as drift with the recommendation that matters - if the shell misbehaves after a Parrot upgrade, rebuild the private stack with install.sh --start-after quickshell, which costs minutes, not hours, because the system dependency set is untouched.

> **Why not pin Qt with apt preferences?.** Pinning would freeze security fixes for the most exposed GUI library on the machine. The architecture already isolates Caelestia's own binaries in /opt; what flows from apt is shared system infrastructure, and it should keep receiving Debian security support. Drift detection gives the same early warning without the risk.

## 8. Orchestration Runtime and Failure Semantics

install.sh is a stage machine under strict shell mode (set -Eeuo pipefail, inherit_errexit) with exactly one cleanup authority: the EXIT trap. Every failure path - a failed command, an explicit die, an interrupt - funnels into that trap with the failing stage recorded, and the trap either reports success or rolls the machine back to the pre-run state.

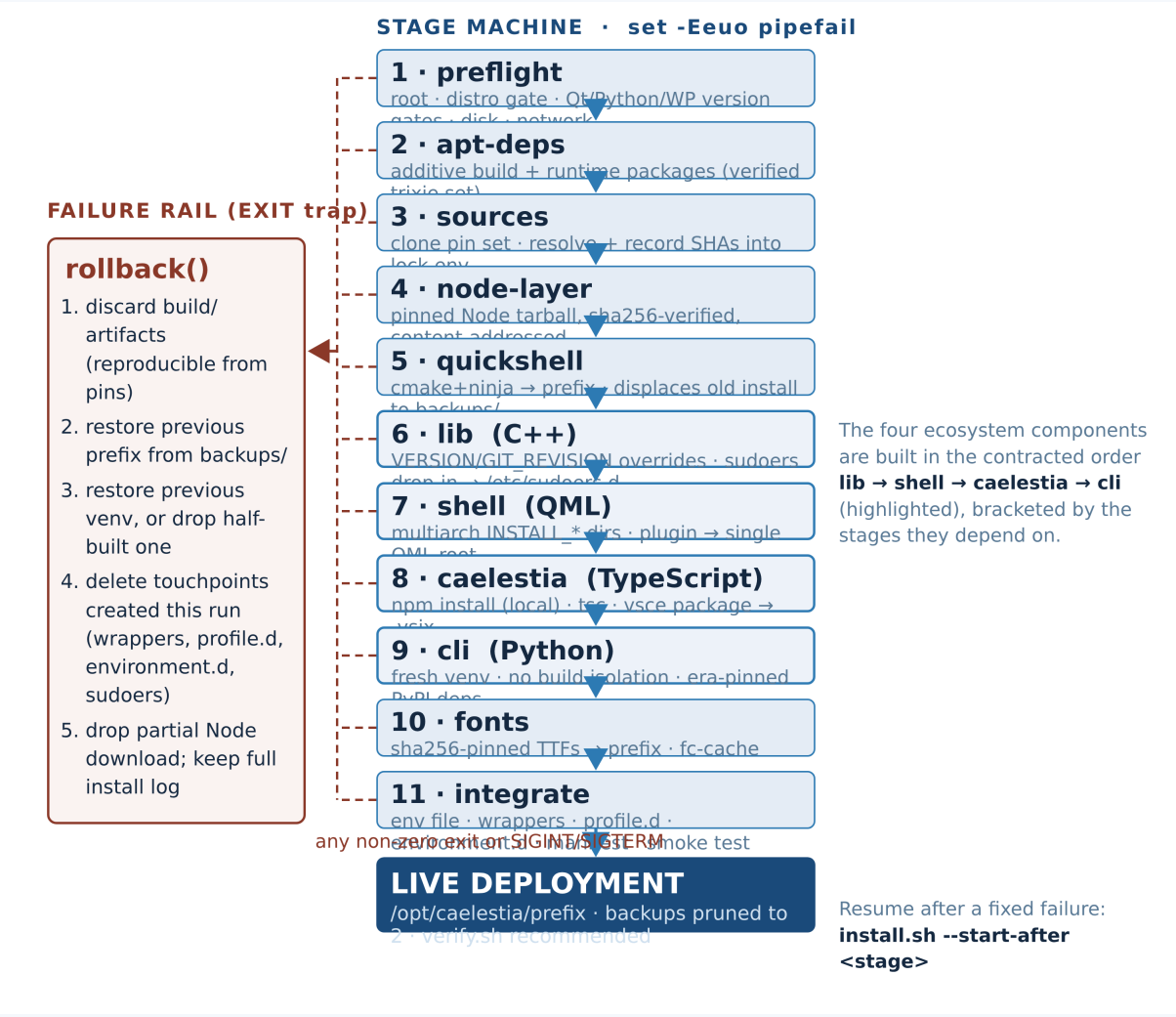



*Figure 2 - The eleven-stage pipeline with its failure rail. Any non-zero exit or signal routes every stage into the rollback handler; the four ecosystem components build in the contracted order lib -> shell -> caelestia -> cli.*

| Stage | Scope | On failure |
|---|---|---|
| 1 preflight | root check, distro gate, Qt/Python/protocol version gates, disk, network | clean exit, nothing to undo |
| 2 apt-deps | additive install of the verified package set | packages remain (additive-only by design; nothing was replaced) |
| 3 sources | clone pin set, resolve and record SHAs | partial clones discarded next run |
| 4 node-layer | pinned Node tarball, SHA-256 verified | partial download removed |
| 5 quickshell | first prefix-mutating stage; moves previous tree to backups/ | full rollback: prefix + venv restored, touchpoints deleted |
| 6-9 lib / shell / caelestia / cli | the four components in the contracted order | same full rollback |
| 10 fonts | pinned TTF downloads into the prefix | same |
| 11 integrate | env file, wrappers, profile.d, environment.d, manifest, smoke test | same; smoke-test failure means the deployment never goes live broken |

*Table 8 - Stage semantics and rollback coverage.*

Two engineering details are worth calling out because they are easy to get wrong. First, the backup-first cutover: the installer moves any existing prefix to backups before building the new one in place, because CMake bakes absolute RPATH and QML paths into binaries and a rename-a-staging-tree scheme would leave every binary pointing at a directory that no longer exists. Second, signal discipline: INT, TERM and HUP handlers convert interrupts into exit code 130, which the EXIT trap treats identically to any build failure - so Ctrl-C during a compilation leaves a consistent machine, a full log, and a resume command printed on the console (--start-after ).

# 9. Security and Supply Chain

An installer that runs as root, downloads toolchains and builds compilers' worth of code is itself an attack surface. The design accepts that and constrains it: every byte that enters the machine is either fetched over TLS from a pinned location and checksummed, or cloned from a pinned git ref whose resolved SHA is recorded.

- **Pinned downloads:** the Node.js tarball and both fonts carry SHA-256 digests in the script header; a mismatch aborts the install before anything is unpacked.
- **Pinned sources:** tags and raw commit SHAs only, resolved and recorded in lock.env - never branch heads, never 'latest'.
- **No remote execution:** nothing is ever piped from the network into a shell; the only executed code is this script and the sources it just built.

- **Scoped npm:** cache, fund and audit settings point inside /opt; no global installs, no telemetry, no state in $HOME.
- **Scoped pip:** the venv is rebuilt from pinned constraints with build isolation disabled, so the wheel build cannot silently pull a different backend.
- **Scoped sudo:** the only privileged file is the theming sudoers rule, which permits exactly two helper paths (the Chromium policy helper and papirus-folders) and nothing else; it is chmod 0440, visudo-validated, and skippable with --without-sudoers.
- **Never touched:** $HOME of any user, /usr/lib, the dpkg database, and the system Python environment.

# 10. Verification and Diagnostics

Verification is layered the same way the stack is: files, linkage, versions, then live behavior. All of the commands below are what verify.sh automates, but they are documented here in manual form because a field engineer needs to be able to run them one at a time on a broken machine.

## 10.1 Static checks (no graphical session needed)

**Static verification commands**

```
# 1. Layout and multiarch pathing
ls /opt/caelestia/{env,lock.env,manifest.env}
ls /opt/caelestia/prefix/lib/$(gcc -dumpmachine)/ # private libs
ls /opt/caelestia/prefix/lib/$(gcc -dumpmachine)/qt6/qml/
# -> Caelestia/ Quickshell/ (single QML root)
# 2. Linkage: everything resolves, private libs come from the prefix
ldd /opt/caelestia/prefix/lib/$(gcc -dumpmachine)/libcaelestia.so.* \
| grep -c 'not found' # -> 0
ldd /opt/caelestia/prefix/lib/$(gcc -dumpmachine)/libcaelestia.so.* \
| grep /opt/caelestia # -> private libs served from the prefix
# 3. Wrappers and CLI respond
caelestia --version
qs --help >/dev/null && echo qs-ok
```

## 10.2 Live smoke test (from inside the Hyprland session)

The real proof that the QML shell, its plugin and the CLI agree with each other is an IPC round trip through quickshell. Run the following as your normal user after logging into Hyprland:

**Live verification commands**

```
# Start the shell detached (what the dotfiles would autostart)
caelestia shell -d
```

```
# Confirm the process and its QML world loaded cleanly
pgrep -af 'qs -c caelestia' # -> running quickshell instance
# IPC round trip 1: raw quickshell interface
qs -c caelestia ipc show
# IPC round trip 2: the same trip through the Python CLI
caelestia shell --show
# Foreground debug run when something is wrong (live QML log)
qs -c caelestia -n
# Stop it again
caelestia shell --kill
```

| Symptom | Most likely cause | Fix |
|---|---|---|
| qs: command not found | session predates the install; PATH lacks the wrapper | log out/in, or: . /opt/caelestia/env |
| module 'Caelestia' is not installed | QML2_IMPORT_PATH not set in the launching environment | start via the caelestia/qs wrappers; check environment.d file |
| libcaelestia.so.1: cannot open shared object | private libdir moved or arch changed | re-run installer; check gcc -dumpmachine matches install arch |
| IPC call returns nothing | shell not running, or quickshell generation mismatch | start shell; re-check the pin set in lock.env |
| verify.sh reports Qt drift after apt upgrade | system Qt was upgraded since install | expected and safe within 6.8.x; rebuild private stack if errors appear |

*Table 9 - Diagnostic interpretation matrix.*

verify.sh wraps all of this into a single pass with three exit codes: 0 healthy, 2 warnings only, 1 failures. The drift check at the end compares the live system against manifest.env, which is the early-warning system for Parrot upgrades: it tells you what changed underneath the deployment before users notice anything.

# 11. Operational Runbook

Day-to-day operation needs exactly three commands: start the shell, verify health, and occasionally rebuild. This chapter adds the two procedures that need judgment - applying the dotfiles, and moving to a newer Caelestia generation - plus uninstallation.

## 11.1 Applying the dotfiles (deliberately manual)

The installer never writes into any user's home directory. The pinned dotfiles tree sits at /opt/caelestia/src/caelestia; applying it is a per-user decision. At this era the upstream procedure is a manual copy of the hypr directory (plus foot, fish and the rest) into the corresponding XDG configuration locations - the repository README of the pinned commit documents the exact list. Keeping this step manual is a feature: dotfiles carry keybinds and application choices that belong to the person at the keyboard, not to the machine.

## 11.2 Upgrading the deployment

Two different clocks drive upgrades. The system clock: Parrot ships security updates continuously, and the deployment absorbs them without action (verify.sh confirms this with its drift report). The ecosystem clock: to move to a newer Caelestia generation, edit lock.env deliberately - bump the shell pin past v1.2.0 only when the system Qt is 6.9 or newer (the preflight gate will refuse otherwise and tell you why) - then re-run install.sh. The build is minutes, the previous prefix is retained in backups/, and a bad generation is one mv command away from being undone.

## 11.3 Uninstall

sudo install.sh --uninstall removes /opt/caelestia in full and every tracked touchpoint with it (it asks for confirmation unless --yes). The apt packages installed by the apt-deps stage are deliberately left in place: they are stock trixie packages, additive by definition, and another package on the machine may legitimately use them. Operators who want them gone can remove them with apt in the usual way.

## 11.4 The private-Qt future option

When Parrot's Qt reaches 6.9+ the pin set can simply be bumped. Before that day, the only route to current-generation Caelestia is a private Qt under /opt/caelestia/qt built from the qtbase source tarball with the same hardening contract, all C++ components reconfigured with CMAKE_PREFIX_PATH pointing at it, and the env file extending LD_LIBRARY_PATH and QML2_IMPORT_PATH accordingly. It is roughly a one-hour unattended build on four cores. This blueprint deliberately does not ship it: maintaining a private toolkit contradicts the use-Debian's-standard-libraries requirement, and the frozen era already delivers a complete, stable desktop.

# Appendix A: install.sh Usage Reference

The complete command-line surface of the orchestrator, for operators and for inclusion in internal runbooks.

| Option | Effect |
|---|---|
| (no options) | full install: all eleven stages in order |
| --doctor | run /opt/caelestia/verify.sh (the diagnostic suite) |
| --uninstall [--yes] | remove /opt/caelestia and all tracked touchpoints (asks unless --yes) |
| --source-dir PATH | use local checkouts at PATH/{lib,shell,cli,caelestia,quickshell} instead of cloning |
| --ref COMP=REF | override one pin for this run and persist it in lock.env (COMP: lib, shell, cli, qs, dots) |
| --start-after STAGE | resume after a fixed failure (STAGE is printed by the failure message) |
| --stop-after STAGE | stop after a named stage (partial installs, debugging) |
| --with-hyprland | also install Hyprland + portals from apt; prints backports instructions when unavailable |
| --without-sudoers | skip the theming sudoers drop-in |
| --no-fonts | skip the pinned font downloads |
| --keep-build | keep build/ artifacts after a failed run (default: rollback discards them) |
| --force | bypass the distro gate and the Qt-6.9 pin guard (expert use) |

*Table A1 - install.sh options.*

**Quick reference**

```
Exit codes: 0 = success 1 = failure (rolled back) 130 = interrupted
Stage names for --start-after / --stop-after:
preflight apt-deps sources node-layer quickshell lib shell
ts-integrations cli fonts integrate
Logs: /opt/caelestia/logs/install-.log
Lock file: /opt/caelestia/lock.env (the pin contract)
Manifest: /opt/caelestia/manifest.env (system snapshot)
```

# Appendix B: Lock File and File Inventory

The lock file is intentionally human-scale: plain shell assignments, one line per pin, commented. This is the entire compatibility surface of the deployment - if you change nothing here, re-running the installer reproduces the same machine state.

**lock.env format**

```
# /opt/caelestia/lock.env - generated by install.sh, edit deliberately
LIB_REF="d8f3b72b7bffa22ace30595c3190a9f8cedb7e44"
SHELL_REF="v1.1.1"
CLI_REF="8bc7e495af42c778143771179d23312f8ff7a3c8"
QS_REF="v0.2.1"
DOTS_REF="1a69a48730842c5e77b746d751ac337edd20b796"
NODE_VERSION="v26.13.0"
# resolved - audit trail of exact built SHAs
LIB_SHA="..." QS_SHA="..." SHELL_SHA="..." ...
```

| Path | Owner | Removed by |
|---|---|---|
| /opt/caelestia (entire tree) | install.sh | --uninstall |
| /usr/local/bin/caelestia, /usr/local/bin/qs | installer (tracked) | --uninstall / rollback |
| /etc/profile.d/caelestia.sh | installer (tracked) | --uninstall / rollback |
| /etc/environment.d/50-caelestia.conf | installer (tracked) | --uninstall / rollback |
| /etc/sudoers.d/caelestia-theming | installer (tracked, 0440) | --uninstall / rollback |
| ~//.config, ~/.local/share | never written by installer | n/a (dotfiles applied manually per user) |
| apt packages (build + runtime set) | dpkg | left in place by design; removable via apt |

*Table B1 - Complete file inventory of a deployment.*